

Observability & Analytics Stack

Epic	Observability & Analytics MVP
Project	Dreamer — Jungian Dreamwork App
Author	Dmytro Davydov
Status	Draft
Version	1.0
Created	May 15, 2026
Target Release	Post-MVP · Phase 1
Stack	React · Vite · TypeScript · Firebase · PostHog · Sentry
Free-tier only	Yes — PostHog Cloud free + Sentry free. Zero infra cost.

1. Overview

This document defines the requirements for adding a production-ready observability and analytics stack to the Dreamer app. The app is currently deployed with no error monitoring, no product metrics, and no AI performance visibility. This gap makes it impossible to know whether the core dreamwork pipeline is working, whether users are activating, or whether AI latency is within acceptable bounds.

The entire stack described here is implementable on **free tiers** — PostHog Cloud (1M events/month free) and Sentry Browser (5K errors/month free). There is no backend to instrument; the app is client-orchestrated and Firestore-first, so no Datadog APM or server-side agents are required.

Guiding Principle

Dreamer is a privacy-first app. Dream text, journal entries, association text, and element labels are **never** sent to any analytics or monitoring service — only counts, durations, enums, and booleans. This is a hard, non-negotiable constraint enforced at the type level in the analytics service.

2. Problem Statement

Current State

- No error monitoring: bugs surface only when users report them.
- No product funnel: unknown what % of users complete the full dreamwork flow.
- No AI performance data: latency, token cost, and failure rates are invisible.
- No retention signal: no way to know if users return after their first dream.

- No activation metric: the 'aha moment' (first completed dream) is not tracked.

Impact

- Cannot prioritise which pipeline stage to improve without latency data.
- Cannot detect a silent AI failure spike in production.
- Cannot measure whether a product change improved or hurt retention.
- Cannot answer: are users getting value from the app?

3. Goals & Non-Goals

Goals

- Instrument the full activation funnel from dream capture to integration.
- Track AI pipeline health: latency p95, token cost, and failure rate per stage.
- Capture Sentry errors with source maps and alert on AI error spikes.
- Identify users (anonymous UID only) with persistent traits for segmentation.
- Build PostHog dashboards for the activation funnel and AI health.

Non-Goals

- No Datadog, no server-side APM — backendless architecture makes these irrelevant.
- No session recordings containing dream input fields (masked by config).
- No cross-user data aggregation beyond aggregate event counts.
- No paid tier features in scope for the initial implementation.

4. Stack Selection

Tool	Purpose	Free Tier Limit	Why This Tool
PostHog Cloud	Product analytics, funnels, retention, AI metrics dashboards	1M events / month Unlimited dashboards Funnels & retention included	Client-side SDK; privacy-safe config; built-in funnel & retention views; no backend needed
Sentry Browser	Error monitoring, stack traces, release tracking, alerts	5K errors / month 10K perf units / month 30-day retention	Industry standard; source-map support via vite-plugin; beforeSend hook strips request bodies

Both tools operate entirely client-side. No backend proxy, no additional infrastructure, and no ongoing cost at MVP scale.

5. Event Schema

All events are typed via a discriminated union in **src/services/analytics/types.ts**. No free-form event names or property bags — compile-time safety enforced.

Layer 1 — User Lifecycle

Event	Properties	Metric Enabled
user_created	authMethod: 'anonymous'	New user volume, anonymous auth adoption
disclaimer_accepted	sessionNumber	Disclaimer completion rate

Layer 2 — Session & Activation

Event	Properties	Metric Enabled
session_started	sessionNumber, totalDreamsOnRecord	DAU, session frequency
dream_capture_started	sessionNumber	Intent-to-dream rate
dream_submitted	wordCountBucket, hasLifeContext, hasMood	Capture completion rate
first_dream_completed	minutesFromCaptureToDone	Activation rate, time-to-value

Layer 3 — Feature Interactions

Event	Properties	Metric Enabled
structuring_triggered	dreamId	Structuring adoption
element_edited / deleted	elementKind	User override rate by kind
association_added	emotionalValence, salience (1–5)	Association depth
interpretation_triggered	associationCount, elementCount	Interpretation reach
hypothesis_feedback_given	lens, feedback, hypothesisIndex	Resonance rate by Jungian lens
integration_triggered	—	Integration funnel step
journal_saved	wordCountBucket	Journal engagement
dream_history_opened	totalDreamsOnRecord	Return/replay rate

Layer 4 — AI Pipeline Events

Event	Properties	Metric Enabled
ai_stage_completed	stage (extract interpret integrate), latencyMs, promptTokens, completionTokens, model, success	p95 latency per stage, daily token cost, cost per DAU
ai_stage_failed	stage, errorKind (from LlmErrorKind), latencyMs	Failure rate per stage, error type breakdown

6. Privacy Constraints

These rules are absolute and apply to every event in perpetuity. They are enforced at the TypeScript type level so violations cause compile errors, not runtime leaks.

Prohibited Properties (never send to any analytics tool)

- `rawText`, `journalText`, `associationText`, `hypothesisText` — dream content fields
- `element label` — could reveal dream content
- `mood (string)`, `lifeContext (string)` — free-text fields
- Any property derived from the above

Permitted Properties

- Counts (as buckets: `<50` | `50-150` | `150-500` | `500+` for word counts)
- Durations in milliseconds
- Enums: `status`, `lens`, `emotionalValence`, `elementKind`, `errorKind`
- Booleans: `hasLifeContext`, `hasMood`, `success`
- Numeric ratings: `salience (1–5)`, `hypothesisIndex (0–2)`
- Firebase Anonymous UID (opaque identifier — no email, name, or device fingerprint)

Sentry Configuration

The **beforeSend** hook must delete **event.request.data** before any event is transmitted. Session recordings must have **maskAllInputs: true** in PostHog to prevent dream text from appearing in replays.

7. File Structure

New files to create under **src/services/analytics/**:

```
src/services/analytics/  
  client.ts      # PostHog init + typed capture() + identify() wrappers  
  types.ts       # Discriminated union of all event payloads  
  sentry.ts      # Sentry init + captureError() wrapper  
  performance.ts # AI stage timer: startAiTimer() → stop()  
  index.ts       # Re-exports
```

Both Sentry (**initSentry()**) and PostHog (**initAnalytics()**) are initialised in **main.tsx** before the React tree mounts — before **initFirebase()** and **ensureAnonymousAuth()**.

8. Integration Points

File	Events to Wire	Priority
main.tsx	initSentry(), initAnalytics()	High
app/config/firebase.ts (after auth resolves)	identifyUser(uid, traits), user_created (first session only)	High
features/dreamCapture/ui/DreamEntryPage.tsx	dream_capture_started, dream_submitted	High
features/dreamStructuring/service/extractElements.service.ts	ai_stage_completed / ai_stage_failed (extract)	High
features/dreamInterpretation/service/generateHypotheses.service.ts	interpretation_triggered, ai_stage_completed / failed (interpret)	High
features/dreamIntegration/service/generateIntegration.service.ts	integration_triggered, ai_stage_completed / failed (integrate)	High
features/dreamStructuring/ui/DreamBreakdownPage.tsx	element_edited, element_deleted, element_added_manually	Medium
features/dreamAssociations/ui/AssociationsPage.tsx	association_added	Medium
features/dreamInterpretation/ui/InterpretationPage.tsx	hypothesis_feedback_given	Medium
features/dreamIntegration/ui/DreamIntegrationPage.tsx	journal_saved	Medium
screens/DashboardPage.tsx	session_started, dream_history_opened	Medium

9. Environment Variables

Add to **.env.example** and populate in the deployment environment:

```
VITE_POSTHOG_KEY=phc_...  
VITE_SENTRY_DSN=https://...@sentry.io/...  
VITE_APP_VERSION=0.1.0
```

Keys are never committed to source control. The analytics service performs a guard check — if **VITE_POSTHOG_KEY** is absent (e.g., in test environments), all capture calls are no-ops.

10. PostHog Dashboards to Configure

Activation Funnel

dream_capture_started → dream_submitted → structuring_triggered → interpretation_triggered → integration_triggered → first_dream_completed

AI Health Dashboard

- p95 latency per stage (extract / interpret / integrate) — 7-day rolling
- Failure rate per stage over 7 days — kill threshold: >5%
- Daily token spend (promptTokens + completionTokens) per stage
- Cost per DAU estimate (token spend / unique users)

Engagement Signals

- hypothesis_feedback_given rate — what % of interpretation users rate hypotheses
- journal_saved rate — what % of users complete the integration step
- dream_history_opened as a return-visit / retention proxy

Retention (PostHog Cohorts)

D1, D7, D14 retention cohorts seeded from **user_created**. Available on the PostHog free tier under the Retention view.

11. Sentry Alerting

Alert	Condition	Channel	Priority
AI error spike	ai_stage_failed error rate > 3% in 1-hour window	Email / Slack	High
Unhandled exception surge	> 50 new errors in 10 minutes	Email	High
New issue introduced in release	First occurrence of any error in a new release	Email	Medium

12. Implementation Order

Phase	Work	Files Touched	Priority
1 — Plumbing	Install posthog-js + @sentry/browser + @sentry/vite-plugin; create analytics service files; wire main.tsx; add env vars	main.tsx, vite.config.ts, .env.example, src/services/analytics/*	High
2 — AI Observability	Wrap callLlm() calls in the three AI service files with startAiTimer(); emit ai_stage_completed / ai_stage_failed	extractElements.service.ts, generateHypotheses.service.ts, generateIntegration.service.ts	High
3 — Activation Funnel	Wire capture() calls into DreamEntryPage, DreamBreakdownPage, InterpretationPage, DreamIntegrationPage	4 feature UI files	High
4 — User Identity	Call identifyUser() after anonymous auth resolves; emit user_created on first session	app/config/firebase.ts	Medium
5 — Dashboards & Alerts	Build PostHog activation funnel + AI health dashboard; configure Sentry alert rules	PostHog UI, Sentry UI	Medium

13. Success Criteria

Criterion	Target	Priority
All 4 AI stage events firing in production	Day 1 post-deploy	High
Activation funnel visible end-to-end in PostHog	Week 1	High
AI p95 latency visible per stage	Week 1	High
AI failure rate below baseline threshold	< 3% per stage	High
Zero dream content in Sentry or PostHog	Always — verified by manual event audit	High
Sentry alert firing on AI error spike	Within 60 min of threshold breach	Medium
D7 retention cohort visible in PostHog	After 7 days of production traffic	Medium

14. Approvals

Role	Name	Signature	Date
Engineering Lead			
Product Owner			